DESARROLLO DE APLICACIONES II

Definiciones técnicas de la plataforma de eventos

Equipo 9 — Core · Desarrollo de Aplicaciones II

27 de octubre de 2026

Contenido

1. Tecnología definitiva 3
2. Tópicos, colas y canales 3
3. Registro de productores y consumidores 4
4. Formato oficial del sobre 4
5. CloudEvents 5
6. Autenticación entre módulos 5
7. Claims y validación de JWT 6
8. Reintentos y confirmación de recepción 6
9. Dead Letter Queue 7
10. Idempotencia y deduplicación 8
11. Versionado y compatibilidad de contratos 8
12. Observabilidad, trazabilidad y auditoría 8
13. ⚠️ URLs de los ambientes — a definir 9
14. ⚠️ Entrega segura de credenciales — a definir 9
Resumen 10

Respuesta del equipo Core (M9) a la consulta de 14 puntos.

> **12 de los 14 están definidos y se pueden implementar hoy.** Los dos que faltan —las URLs de los ambientes y la entrega de credenciales— dependen de un despliegue que todavía no hicimos. Los marcamos como ⚠️ para que nadie implemente contra algo que no existe.

---

1. Tecnología definitiva

RabbitMQ 4.3. No Azure Event Grid, no Kafka.

Por qué	
DLQ nativa	`x-dead-letter-exchange` es parte del broker, no hay que construirla.
Backoff sin scheduler	Una cola por escalón con `x-message-ttl` y dead-letter de vuelta al exchange: el retraso lo hace el broker, no un proceso durmiendo.
Corre local	`brew install rabbitmq` y cualquiera levanta la plataforma entera en su máquina. Con un servicio gestionado en la nube, nadie puede desarrollar sin credenciales.
Consola de administración	Ver colas y mensajes a ojo durante la integración vale mucho con 9 equipos.

Kafka da retención y replay, que son mejores para auditoría, pero **DLQ y reintentos hay que implementarlos a mano**. Para un hub cuyo requisito central es «los mensajes fallidos no se pierden», RabbitMQ lo trae resuelto.

La decisión está tomada y el Core ya está implementado sobre ella. Cambiarla ahora implicaría rehacer la capa de mensajería y la topología.

> El código no está atado a RabbitMQ: los servicios dependen de una interfaz `Broker` (patrón Adapter) con dos implementaciones. Agregar Kafka sería escribir un `KafkaBroker` sin tocar lógica de negocio. Pero no está en los planes.

2. Tópicos, colas y canales

Para publicar — uno solo, compartido

```
muni.inbox        fanout, durable
```

Todos los módulos publican ahí. **No eligen destino**: el ruteo lo decide el Core leyendo las suscripciones.

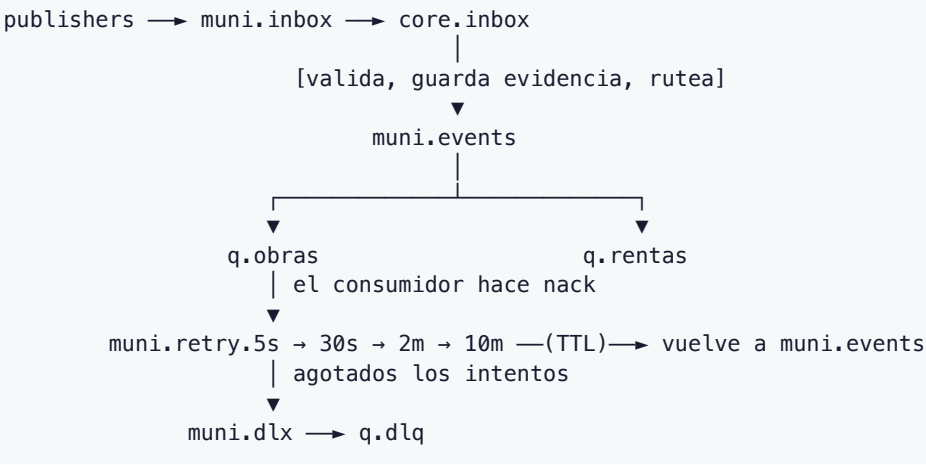
Para recibir — una cola por módulo

```
q.ciudadanos    q.atencion-ciudadana    q.obras      q.habilitaciones
q.rentas        q.ambiente              q.transito   q.desarrollo-social
```

En su cola les llega **solo lo que pidieron**, no todo el tráfico. Pero es **una cola con varios tipos adentro**, no una por tipo: hay que despachar por `eventType`.

Internos del Core — no se tocan

Exchange	Para qué
`muni.events`	Por donde el Core reparte. Routing key = nombre de la cola destino.
`muni.retry.15s` · `1m` · `5m` · `15m`	Los escalones de backoff.
`muni.dlx` → `q.dlq`	Lo que agotó los reintentos.



3. Registro de productores y consumidores

Cada equipo se autoadministra, no hace falta pedirle nada al Core.

```
POST   /api/v1/publications    { "eventType": "workOrderCompleted" }
POST   /api/v1/subscriptions   { "eventType": "ticketCreated", "maxAttempts": 4 }
DELETE /api/v1/subscriptions/{id}
GET    /api/v1/event-types/map        quién publica y quién consume cada tipo
```

Al crear una suscripción, **el Core declara la cola y su binding en el broker en el acto**. No hay que coordinar nada antes.

> Declaren su cola en **modo pasivo** (`passive=True` / `checkQueue`). La crea el Core; si la declaran con argumentos distintos, RabbitMQ cierra el canal con `PRECONDITION_FAILED` .

Declarar una publicación es **documentación, no un permiso**: publicar un tipo no declarado funciona igual. Existe para que el mapa detecte los agujeros de integración.

4. Formato oficial del sobre

```
{
  "eventId": "3f6c1b7e-9d24-4a1f-9f2a-2b0f0c7d5e11",
  "eventType": "workOrderCompleted",
  "eventVersion": "1.0",
  "occurredAt": "2026-09-29T18:00:00Z",
  "sourceModule": "obras",
  "correlationId": "8a1f0c22-5d3e-4b77-9c10-6e2b4a90f3d5",
  "causationId": null,
  "data": {}
}
```

Campo	Tipo	Oblig.	Regla
`eventId`	UUID v4	**sí**	Nuevo por evento. Clave de idempotencia.
`eventType`	string ≤120	**sí**	Inglés, camelCase.
`eventVersion`	string ≤20	no (`"1.0"`)	Versión del contrato.
`occurredAt`	ISO-8601	**sí**	**Con offset obligatorio.**
`sourceModule`	string ≤60	**sí**	El nombre registrado del módulo.
`correlationId`	UUID	no	Hilo del trámite, se propaga.
`causationId`	UUID	no	El `eventId` que disparó este.
`data`	objeto	**sí**	Libre.

No se aceptan campos extra (`additionalProperties: false`): un campo no previsto se rechaza con `422` en vez de ignorarse. Todo lo propio va en `data` .

Schema en vivo: `GET /api/v1/events-meta/envelope-schema` .

5. CloudEvents

No lo usamos de forma nativa. El sobre es propio.

La razón es de oportunidad, no técnica: los 9 equipos ya tienen el envelope acordado y hay código escrito contra él. Cambiarlo ahora rompería a todos por un beneficio que, puertas adentro de una plataforma cerrada, es chico.

El mapeo es 1:1, así que si usan un SDK de CloudEvents pueden adaptar en su borde sin problema:

CloudEvents 1.0	Nuestro sobre	Nota
`id`	`eventId`	
`source`	`sourceModule`	Nosotros usamos el nombre del módulo, no un URI.
`type`	`eventType`	Sin prefijo de dominio invertido.
`time`	`occurredAt`	Mismo formato RFC 3339.
`data`	`data`	Idéntico.
`datacontenttype`	—	Siempre `application/json` , implícito.
`dataschema`	—	El JSON Schema vive en el catálogo, por tipo de evento.
`specversion`	—	No lo emitimos.
`subject`	—	No lo usamos.
extensión	`correlationId` , `causationId` , `eventVersion`	CloudEvents permite atributos de extensión.

Si al grupo le sirve, podemos evaluar emitir CloudEvents **además** del formato actual, pero habría que acordarlo entre los 9 equipos, no solo con el Core.

6. Autenticación entre módulos

Los módulos **no se autentican entre sí**: nadie llama a nadie directamente. Todo pasa por el broker, y el broker autentica con las credenciales de RabbitMQ.

Para llamar a la **API del Core** (publicar por HTTP, administrar suscripciones, ver el dashboard) sí hace falta un token:

```
POST /api/v1/auth/module-token
{ "module": "obras", "secret": "<secret de maquina>" }
```

```
{ "accessToken": "eyJ...", "tokenType": "Bearer", "expiresIn": 900, "kind": "module" }
```

Dura **15 minutos** y no hay refresh: cuando expira se pide otro. El secret vive en la configuración del backend de cada equipo y no lo usa ninguna persona.

> Hay una segunda credencial, distinta: **email + contraseña por integrante**, para entrar al dashboard. Están separadas a propósito: rotar el secret de máquina no le corta el acceso al dashboard a nadie, y cambiar la contraseña de alguien no toca el backend desplegado.

7. Claims y validación de JWT

Algoritmo RS256. Claims del token de módulo:

Claim	Ejemplo	Qué es
`sub`	`module:obras`	El sujeto.
`module`	`obras`	El módulo. Es el que se compara con el `sourceModule` al publicar.
`kind`	`module`	`module` (backend) o `user` (persona en el dashboard).
`isAdmin`	`false`	Solo el equipo 9. Habilita ver el tráfico de todos.
`displayName`	`Obras Publicas`	Para mostrar.
`iss`	`muni-core`	Emisor.
`aud`	`muni-platform`	Audiencia.
`iat` · `exp`	epoch	Emisión y expiración (15 min).
`jti`	—	Identificador del token.

Quién valida

Solo el Core. Los módulos no necesitan validar nada: los tokens que emitimos sirven para llamarnos a nosotros, y ustedes nunca reciben uno.

Por eso hoy **no exponemos un JWKS**. Si en algún momento un módulo necesita validar tokens del Core sin llamarnos, lo agregamos — es publicar la clave pública en `/.well-known/jwks.json` y es un cambio chico. Pero hoy no haría falta y preferimos no mantener superficie que nadie usa.

8. Reintentos y confirmación de recepción

Confirmación

Situación	Qué hacer
Procesado bien	`ack`
Error transitorio (su base cayó, timeout)	`nack(requeue=false)`
Error permanente (el payload no sirve)	`nack(requeue=false)`

Nunca `requeue=true`. La cadena de reintentos la gobierna el Core; si reencolan, el mensaje gira en su cola sin backoff y sin quedar auditado.

Reintentos

> **Corregido el 2026-10-27.** Antes decía `15 s → 1 m → 5 m → 15 m`. Era un `.env` local de demo, no lo desplegado. Perdón por el dato equivocado.
>
> **No copien estos números a un documento propio.** Consúltenlos en `GET /api/v1/topology`, que devuelve cada cola de espera con su `x-message-ttl`. Si no coincide con esto, gana el endpoint.

La cadena de esperas configurada es:

```
5 s → 30 s → 2 min → 10 min → Dead Letter Queue
```

Configurable por suscripción con `maxAttempts` (1 a 10, default 4).

Ojo con qué cuenta `maxAttempts`: entregas, no reintentos. Con el default de 4 son la entrega original más 3 reintentos, y el cuarto `nack` agota. O sea que **el escalón de 10 min no se usa** y la ventana real es de **2 min 35 s**. Para que entre el cuarto escalón hace falta `maxAttempts = 5` (12 min 35 s).

El mecanismo: el Core publica el mensaje en la cola de espera del escalón, que tiene un `x-message-ttl` y su `x-dead-letter-exchange` apunta de vuelta a `muni.events`. Al vencer el TTL, RabbitMQ lo devuelve **conservando la routing key original**, o sea la cola del módulo. **No hay ningún proceso durmiendo.**

Hay una cola por escalón y no una sola con TTL por mensaje porque las colas AMQP son FIFO: con una sola, un mensaje esperando 15 minutos en la cabeza bloquearía a los que esperan 15 segundos detrás.

Publicación

Los mensajes se publican como **persistentes** (`delivery_mode=2`) sobre colas **durables**: sobreviven a un reinicio del broker. Usamos **publisher confirms**.

9. Dead Letter Queue

Lo que agota los reintentos va a `q.dlq`, el Core lo persiste y **nunca lo descarta**.

```
GET  /api/v1/dlq?targetModule=obras&status=OPEN
GET  /api/v1/dlq/{id}             detalle, payload crudo e historial
POST /api/v1/dlq/{id}/retry       reintentar
POST /api/v1/dlq/retry-bulk       hasta 200 de una
POST /api/v1/dlq/{id}/discard     descartar, con motivo obligatorio
GET  /api/v1/dlq/{id}/audit       quién reintentó, cuándo y con qué resultado
```

Motivos posibles:

`reasonCode`	Qué pasó	Cómo se resuelve
`SCHEMA_VIOLATION`	El `data` no cumple el schema declarado.	Se corrige el schema o el evento y se reprocesa el que ya está guardado.
`DELIVERY_FAILED`	El módulo destino agotó sus intentos.	Cuando vuelve, se republica en su cola.
`MALFORMED_MESSAGE`	Ni siquiera era JSON válido.	**No se puede reintentar**, hay que descartarlo con motivo.

El reintento manual queda auditado con la **persona** que lo disparó, no solo con el equipo.

10. Idempotencia y deduplicación

Del lado del Core

`eventId` es **UNIQUE** en la bitácora. Si llega repetido, se marca `DUPLICATE` y **no se vuelve a rutear**. La respuesta es `200` con `duplicate: true` — no es un error.

Del lado de ustedes — hace falta igual

La entrega es **at-least-once**. Un mensaje les puede llegar más de una vez:

- un `nack` que se reintenta;
- una caída de conexión después de procesar pero antes del `ack`.

Guarden los `eventId` que ya aplicaron y salgan temprano si se repite. El Core no puede garantizar exactly-once hacia ustedes; ningún broker puede.

11. Versionado y compatibilidad de contratos

`eventVersion` en el sobre, string libre (`"1.0"` , `"1.1"` , `"2.0"`).

Cuándo subir la versión:

Cambio	¿Rompe?
Agregar un campo **opcional**	No. Misma versión.
Agregar un campo **requerido**	Sí, hacia atrás: los eventos ya publicados no lo traen.
Quitar un campo requerido	Sí, hacia adelante: los consumidores viejos lo esperan.
Cambiar el **tipo** de un campo	Sí, en las dos direcciones.
Quitar valores de un `enum`	Sí, hacia atrás.
Agregar valores a un `enum`	Sí, hacia adelante, si el consumidor lo tiene cerrado.

Recomendación: dejen `additionalProperties: true` en sus schemas de `data`. Así pueden agregar campos sin romper a nadie. Si lo cierran, cualquier campo nuevo se vuelve un cambio incompatible.

Los tipos de evento **no se borran**: hay eventos históricos que los referencian. Se marcan `DEPRECATED` y quedan avisando en el catálogo.

12. Observabilidad, trazabilidad y auditoría

Trazabilidad

Dos identificadores, con propósitos distintos:

	Qué sigue
`correlationId`	**El trámite.** Lo propagan ustedes entre eventos. Reconstruye la journey completa entre módulos.
`traceId`	**La operación técnica.** Lo genera el Core, viaja en el header `X-Trace-Id` y queda en los logs y en la bitácora.

```
GET /api/v1/events/journey/{correlationId}     el recorrido del trámite
GET /api/v1/events/{eventId}                   un evento y el estado de cada entrega
```

Si algo no cierra, pásennos el `traceId`: con eso encontramos el evento y su rastro completo.

Evidencia

El Core guarda **el sobre tal cual llegó** y no lo reescribe nunca, más una fila por cada entrega con su estado, intentos y último error.

Observabilidad

```
GET /api/v1/dashboard                          estadísticas del módulo
GET /api/v1/dashboard/integration-alerts       agujeros de integración detectados
GET /health/ready                              estado del Core y del broker
```

Hay métricas en formato Prometheus y un dashboard de Grafana en el repo.

Alertas de integración

El Core no valida reglas de negocio, pero ve el mapa completo de quién publica qué y quién consume qué. Cruzando esas listas detecta solo:

- eventos que alguien publica y **nadie consume**;
- nombres **sospechosamente parecidos** (`debtOverdue` vs `overdueDebt` , o un typo);
- tipos que **aparecieron sin declararse**.

Hoy encuentra **7 desalineaciones reales** entre los 9 equipos, documentadas en `desalineaciones.md` . Conviene revisarlas antes de implementar.

13. ⚠️ URLs de los ambientes — a definir

No hay nada desplegado todavía. El Core corre en una máquina de desarrollo con `guest:guest@localhost:5672` . No les podemos pasar una URL porque no existe.

Lo decimos derecho para que nadie implemente contra una dirección inventada.

Nuestra propuesta, para cerrarlo rápido:

	Propuesta	Por qué
Dónde	RabbitMQ en Railway	Varios equipos ya lo usan; da control sobre usuarios y vhosts, que los servicios gestionados gratuitos no dan.
Ambientes	**Un servidor, dos vhosts**: `/test` y `/prod`	Un vhost aísla exchanges, colas y mensajes por completo. Un evento de prueba **no puede** llegar a producción. Más barato que dos servidores y la separación es igual de estricta.
Cómo llega	Una sola variable `RABBITMQ_URL`	Con todo adentro, igual que la base de datos.

```
RABBITMQ_URL=amqps://<modulo>:<password>@<host>:5671/test
RABBITMQ_URL=amqps://<modulo>:<password>@<host>:5671/prod
```

Lo que necesitamos: que el grupo confirme la plataforma.

14. ⚠️ Entrega segura de credenciales — a definir

Tampoco está resuelto. Hoy los secrets de máquina se imprimen al correr el seed en la máquina de desarrollo.

Nuestra propuesta:

Un usuario por módulo, no uno compartido	**Si se filtra el de un equipo se rota solo el suyo. Con uno compartido hay que avisarle a los nueve.**
Entrega por canal privado, uno a uno	Nunca en el grupo general ni en un documento compartido.
Rotación sin coordinación	`POST /api/v1/modules/{nombre}/rotate-secret`. El anterior deja de servir en el acto, así que hay que actualizar la config del backend — pero **no afecta el acceso de las personas al dashboard**.
Nunca en el repositorio	Van en las variables de entorno de la plataforma de despliegue.

Si el grupo prefiere un gestor de secretos, lo evaluamos — pero para 9 equipos y un cuatrimestre, variables de entorno más rotación por API alcanza.

Resumen

#	Punto	Estado
1	Tecnología	✅ RabbitMQ 4.3
2	Tópicos y colas	✅ `muni.inbox` para publicar, `q.<modulo>` para recibir
3	Registro	✅ Autoservicio por API
4	Sobre	✅ Definido, schema en vivo
5	CloudEvents	✅ No nativo, con mapeo 1:1 documentado
6	Autenticación	✅ Token de módulo, 15 min
7	Claims JWT	✅ RS256, solo valida el Core
8	Reintentos y ack	✅ 5s → 30s → 2m (con `maxAttempts` default)
9	DLQ	✅ Con reintento y descarte auditados
10	Idempotencia	✅ Del lado del Core; hace falta también del suyo
11	Versionado	✅ `eventVersion` + reglas de compatibilidad
12	Observabilidad	✅ Journey, traceId, dashboard, alertas
13	URLs de ambientes	⚠️ **Falta desplegar**
14	Credenciales	⚠️ **Falta acordar**

Los 12 primeros se pueden implementar hoy. Para los dos últimos necesitamos una reunión corta: con la plataforma confirmada, les pasamos las URLs y las credenciales de cada ambiente el mismo día.

Mientras tanto no hace falta esperarnos: se puede levantar RabbitMQ local (`brew install rabbitmq`) o publicar por HTTP contra `POST /api/v1/events`, que pasa por el mismo pipeline.